

Registro de Atividades — COBIB

Relatório Técnico do Projeto

Versão 8 (v8) · Aplicação Web Progressiva (PWA) client-side

Documento de referência para manutenção e evolução do sistema

Destinado a desenvolvedores humanos e a modelos de IA

Sumário

Para preencher os números de página: no Word, clique com o botão direito sobre esta área e escolha “Atualizar campo” (ou selecione tudo e pressione F9).

TOC \h \o "1-3"

1. Visão geral do projeto

Objetivo. O sistema cria um formulário web para registro diário de produção de trabalho da COBIB (Coordenação de Biblioteca). Cada pessoa registra as atividades que executou, vinculadas a um processo de trabalho e a uma atividade com pontuação definida por complexidade (baixa, média, alta). As chefias aprovam os lançamentos em uma cadeia hierárquica, e um administrador gerencia a configuração do sistema.

Restrição fundamental de arquitetura. O projeto foi concebido para funcionar **sem servidor de aplicação e sem banco de dados remoto**. Todo o código executa no navegador do usuário (Chrome ou Edge) e os dados são persistidos em uma **pasta de rede compartilhada** acessada diretamente pelo navegador. Essa decisão condiciona todo o restante do desenho e deve ser respeitada em melhorias futuras, salvo decisão explícita de migrar para uma arquitetura com backend (ver seção 11).

Tecnologias. HTML5, CSS3 e JavaScript puro (sem framework, sem etapa de build, sem dependências externas em tempo de execução). Persistência via `File System Access API` (gravação em arquivos JSON na pasta de rede) e `IndexedDB / localStorage` (estado local do navegador). Empacotado como PWA (Progressive Web App) com `manifest` e `service worker` para instalação e uso offline.

1.1. Público-alvo deste documento

Este relatório foi escrito para permitir que qualquer pessoa desenvolvedora — ou qualquer modelo de IA — retome o projeto sem conhecimento prévio e implemente melhorias com segurança. Ele descreve a arquitetura, o modelo de dados, o fluxo de cada perfil, as decisões de projeto e suas justificativas, as limitações conhecidas e um roteiro de evolução. Presume-se familiaridade com JavaScript e com conceitos web, mas não com este código.

1.2. Histórico de versões

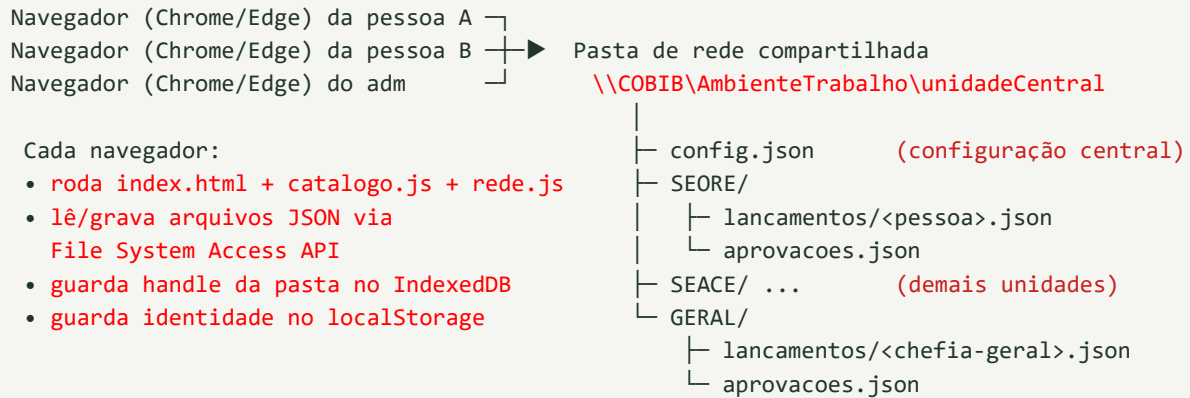
O projeto evoluiu de forma incremental. Conhecer a trajetória ajuda a entender por que certas estruturas existem.

Versão	Marco introduzido
v1	Formulário de lançamento com persistência em localStorage (dados isolados por navegador).
v2	Migração para pasta de rede compartilhada; perfis Servidor, Chefia da unidade e Chefia geral; painel de aprovação.
v3	Chefias passam a registrar atividades; cadeia de aprovação em três níveis (papel gravado em cada lançamento).
v4	Identificação por pessoa única derivada do catálogo (correção de dessincronia de perfil); travamento de identidade.
v5	Senhas opcionais por hash SHA-256; utilitário gerar-senha.html.
v6	Perfil Administrador; migração de pessoas/senhas para config.json central; edição de catálogo em runtime.
v7	Selo de instalação: apenas o administrador oficializa a pasta; validação impede pastas divergentes.
v8	Correção do bug de oficialização (leitura de identidade nula). Versão atual.

2. Arquitetura

O sistema é composto por arquivos estáticos servidos ao navegador. Não há processamento no lado do servidor: um servidor HTTP (ou o próprio duplo clique no arquivo) apenas entrega os arquivos, e toda a lógica — identificação, leitura e gravação de dados, aprovação, administração — executa no cliente. A comunicação de dados entre pessoas acontece exclusivamente por meio de **arquivos JSON gravados em uma pasta de rede comum**, que cada navegador lê e escreve diretamente.

2.1. Diagrama lógico



Obs.: SEORE, SEACE são exemplos de nomes de unidades de trabalho na organização. A pasta de rede compartilhada pode ser alterada para qualquer uma pelo Administrador.

2.2. Princípio “um arquivo, um único escritor”

A regra central que garante integridade sem banco de dados é: **cada arquivo tem um único autor que o grava**. Nenhum arquivo é escrito concorrentemente por duas pessoas. Assim, mesmo com muitos usuários simultâneos em máquinas diferentes, não há condição de **corrida destrutiva** — apenas leituras cruzadas, que são seguras. Concretamente:

- **< Pessoa >.json** (em cada unidade) é gravado somente pelo próprio servidor/chefia dono do arquivo.
- **aprovacoes.json** de uma unidade é gravado somente pela chefia daquela unidade.
- **GERAL/aprovacoes.json** é gravado somente pela chefia geral (aprova as chefias de unidade e a si mesma).
- **config.json** é gravado somente pelo administrador.

Exceção controlada: o administrador pode gravar em arquivos de lançamento de terceiros ao excluir um registro (correção de erro). É uma operação pontual e humana, não concorrente. Ainda assim, é o único ponto onde a regra é relaxada — **melhorias devem tratá-lo com cuidado**.

2.3. Onde cada tipo de estado vive

Estado	Local	Escopo
Lançamentos e aprovações	Arquivos JSON na pasta de rede	Compartilhado (todos)
Configuração (pessoas, senhas, catálogo, selo)	config.json na raiz da pasta	Compartilhado (todos)
Processos e atividades	catalogo.js (arquivo do app)	Semente / estático
Handle da pasta escolhida	IndexedDB (base ra-rede)	Local ao navegador
Identidade confirmada	localStorage (ra.identificacao.v4)	Local ao navegador

Estado	Local	Escopo
Selo aceito neste PC	localStorage (ra.instalacao.v1)	Local ao navegador

3. Estrutura de arquivos do aplicativo

O aplicativo distribuído é um conjunto pequeno de arquivos. A tabela abaixo resume cada um; as seções seguintes detalham os de maior complexidade.

Arquivo	Linhas	Responsabilidade
index.html	~971	Interface (HTML+CSS) e toda a lógica da aplicação em um <script>. É o coração do sistema.
catalogo.js	~148	Dados-semente: processos, atividades, pontuação, unidades, chefias, servidores, senhas iniciais e caminho da pasta.
rede.js	~130	Camada de acesso à pasta de rede: IndexedDB para o handle, permissões, leitura/gravação de JSON.
gerar-senha.html	~63	Utilitário isolado para gerar o hash SHA-256 de uma senha, a ser colado no config.json/catalogo.js.
sw.js	~35	Service worker: cache dos arquivos para funcionamento offline (PWA).
manifest.webmanifest	16	Metadados do PWA (nome, ícones, cores, modo standalone).
icons/	—	Ícones 192px e 512px do aplicativo.
LEIA-ME.txt	~161	Manual operacional para quem for implantar e usar o sistema.

3.1. catalogo.js — dados-semente e configuração inicial

Define constantes globais lidas pelo restante do app. **Importante:** a partir da v6, os campos de **pessoas e senhas** neste arquivo funcionam apenas como **semente** — na primeira oficialização, são copiados para o config.json, que passa a ser a fonte de verdade. O **catalogo.js** permanece autoritativo apenas para **PROCESSOS e ATIVIDADES** (que mudam raramente) e para a constante **PASTA_REDE** (caminho exibido como orientação, necessário antes de haver acesso ao config).

Estruturas exportadas

```
const PONTOS = { b: 5, m: 10, a: 15 }; // baixa, média, alta
const ROTULO_COMPLEXIDADE = { b: 'Baixa', m: 'Média', a: 'Alta' };

const PROCESSOS = [ // 5 processos, 72 atividades
  { nome: '...', atividades: [
    { t: 'texto da atividade', c: ['b','m','a'] }, // c = complexidades permitidas
  ] }, ...
];

const UNIDADES = { // 8 unidades
  'SEORE': { chefia: 'Nome' | null, servidores: ['...'] }, ...
};
const CHEFIA_GERAL = 'Nome' | null;
const SENHAS = { 'Nome': 'hash-sha256', ... }; // semente de senhas
const EXIGIR_SENHA = false;
const PASTA_REDE = '\\\\COBIB\\AmbienteTrabalho\\unidadeCentral';
```

Modelo de complexidade por atividade: o campo c de cada atividade lista quais níveis de complexidade são válidos para ela. O formulário habilita apenas os botões correspondentes e, quando há um único nível, pré-seleciona-o. Há **5 processos e 72 atividades** cadastrados, comuns a todas as 8 unidades (SEACE, SEARI, SECAQ, SEDIN, SENOV, SEORE, SEPERI, SEREN).

3.2. rede.js — camada de persistência

Isola toda a interação com a File System Access API e o IndexedDB. Funções principais:

Função	Papel
salvarHandleRaiz / obterHandleRaiz	Guardam e recuperam o handle da pasta escolhida, no IndexedDB (base ra-rede), para persistir entre sessões.
garantirPermissao(handle, modo, pedir)	Verifica/solicita permissão de leitura ou escrita sobre um handle.
slug(nome)	Converte um nome de pessoa em nome de arquivo seguro (sem acentos/símbolos).
lerJSONDe / gravarJSONEm	Leitura e gravação genéricas de um arquivo JSON dentro de um diretório.
lerMeuArquivo / gravarMeuArquivo	Lançamentos de uma pessoa: <unidade>/lançamentos/<slug>.json.
lerLancamentosDaUnidade	Concatena os lançamentos de todos os arquivos de uma unidade.
lerAprovacoes / gravarAprovacoes	Mapa de aprovações {idLancamento: {por, em}} de uma unidade (ou GERAL).
lerConfig / gravarConfig	Configuração central config.json na raiz.

Detalhe de robustez: a leitura de lançamentos ignora silenciosamente arquivos malformados (try/catch por arquivo), para que um JSON corrompido de uma pessoa não derrube a visão da chefia. Gravações sempre reescrevem o arquivo inteiro (os volumes são pequenos, poucos KB), o que simplifica o modelo e evita edições parciais.

3.3. index.html — interface e lógica

Concentra o CSS (tema visual de inspiração bibliográfica: verde-tinta, dourado de lombada, papel), o HTML das seções e o `<script>` com a lógica. As três seções visuais principais são a etapa 1 (pasta da rede), a etapa 2 (identificação) e, conforme o perfil, o formulário de lançamento, o painel de lançamentos e o painel de administração.

Funções-chave do script principal:

Função	Responsabilidade
conectarSilencioso / aposConectar	Reconecta a pasta memorizada; valida o selo de instalação e define a situação da pasta.
carregarConfig	Lê config.json e popula o objeto global CFG (ou mantém a semente).
oficializarPasta(pessoa)	Grava o selo de instalação (UUID + autor + data) no config.json. Só o adm.
montarPessoas	Constrói a lista do seletor de identidade a partir de CFG; restringe ao adm se a pasta não for oficial.
identificar(pessoa) / limparIdentidade	Efetiva/limpa a identidade; controla travamento e exibição de painéis.
senhaConfere / sha256hex	Validação de senha por hash SHA-256 de (nome + ' ' + senha).
recarregar	Relê os dados da rede conforme o perfil e chama render().
render	Monta a tabela de lançamentos, subtotaís por dia, totais pessoais e resumo por pessoa.
aplicarAprovacao(registros, aprovar)	Grava aprovações no arquivo do nível responsável (unidade ou GERAL).
renderAdm / salvarCFG	Renderiza e persiste o painel de administração (senhas, exigência, catálogo, selo).

4. Modelo de dados (esquemas JSON)

Todos os dados persistidos são JSON. Os esquemas abaixo são o contrato entre as partes do sistema; qualquer alteração neles deve considerar compatibilidade com arquivos já gravados.

4.1. Lançamento (registro individual)

Cada item do array `registros` em `<unidade>/lancamentos/<pessoa>.json`:

```
{
  id:          '1730050000000-a1b2c', // Date.now() + sufixo aleatório (único)
  data:        '2026-07-14',          // ISO aaaa-mm-dd
  unidade:     'SEORE',                // ou 'GERAL' para a chefia geral
  servidor:    'Silvia Regina',        // nome de quem lançou
  papel:       'servidor',             // 'servidor' | 'chefia' | 'geral'
  processo:    'Tratar conteúdo ...',
  atividade:   'Realizar o processamento técnico ...',
  complexidade: 'm',                  // 'b' | 'm' | 'a'
  pontos:      10,                    // derivado de PONTOS[complexidade]
  obs:         '',                    // observações livres (opcional)
  abono:       false                  // lançamento de abono?
}
```

Papel é decisivo: o campo `papel` determina quem aprova o lançamento. Registros de papel `servidor` são aprovados pela chefia da unidade; registros de papel `chefia` ou `geral` são aprovados pela chefia geral. Registros sem o campo (dados anteriores à v3) assumem `servidor` por compatibilidade (função `papelDe`).

4.2. Arquivo de lançamentos de uma pessoa

```
{ versao: 1, unidade: 'SEORE', servidor: 'Silvia Regina',
  atualizadoEm: '2026-07-14T12:00:00.000Z',
  registros: [ { ...lançamento... }, ... ] }
```

4.3. Arquivo de aprovações (`aprovacoes.json`)

Um por unidade e um em GERAL/. Mapa indexado pelo `id` do lançamento:

```
{ versao: 1, unidade: 'SEORE', chefe: 'Nome da Chefia',
  atualizadoEm: '...',
  aprovacoes: {
    '1730050000000-a1b2c': { por: 'Nome da Chefia', em: '2026-07-15T...' },
    ...
  } }
```

A ausência de uma chave significa “pendente”. Aprovar cria a entrada; desfazer a remove. A separação em dois arquivos de aprovação (o da unidade e o GERAL) reflete o princípio do escritor único: a chefia da unidade nunca escreve no arquivo que aprova as chefias, e vice-versa.

4.4. Configuração central (`config.json`)

```
{
  versao: 1,
  atualizadoEm: '...',
  unidades: { 'SEORE': { chefia: 'Nome' | null, servidores: ['...'] }, ... },
  chefiaGeral: 'Nome' | null,
  senhas: { 'Nome': 'hash-sha256', ... },
  exigirSenha: false,
```

```
instalacao: { id:'uuid', por:'Administrador', em:'...' } // selo (v7+)
}
```

5. Perfis, permissões e cadeia de aprovação

Existem quatro perfis. O perfil de uma pessoa é derivado de onde seu nome está cadastrado na configuração — nunca escolhido livremente na tela —, o que elimina inconsistências de identidade (correção introduzida na v4).

Perfil	Registra?	Aprova	É aprovado por	Visualiza
Servidor	Sim	—	Chefia da unidade	Os próprios lançamentos
Chefia da unidade	Sim	Servidores da sua unidade	Chefia geral	Toda a sua unidade
Chefia geral	Sim	Chefias de unidade + a si mesma	Si mesma	Todas as unidades
Administrador	Não	— (não aprova)	— (não é aprovado)	Tudo (para correção)

Autoaprovação da chefia geral: por definição do fluxo, a chefia geral aprova os próprios lançamentos no painel geral. Não há nível acima dela. Já a chefia de unidade **não** consegue aprovar a si mesma — seus lançamentos aparecem como pendentes até a chefia geral agir (a interface só oferece o controle de aprovação a quem tem alçada, via função `podeAprovar`).

Administrador: não registra atividades nem aprova. Suas atribuições são: gerenciar senhas (definir/alterar/remover, inclusive a própria; a senha inicial é `adm`), ativar a exigência global de senha, excluir qualquer lançamento para correção de erros (removendo também a aprovação órfã, se houver) e editar o catálogo de unidades/chefias/servidores. Também é o único que oficializa a pasta central.

5.1. Passo a passo do uso

1. **Conectar a pasta** (etapa 1): a pessoa seleciona a pasta central no navegador; a escolha fica memorizada no IndexedDB.
2. **Validação do selo:** o app lê o `config.json`. Se a pasta não for oficial e a pessoa não for o `adm`, o acesso é barrado.
3. **Identificar-se** (etapa 2): escolha do nome; se houver senha cadastrada (ou for o `adm`), digita a senha. A identidade trava para a sessão.
4. **Operar:** servidor/chefias registram e acompanham; chefias aprovam; `adm` administra.
5. **Atualizar dados da rede:** relê os arquivos para refletir o que outras pessoas gravaram (não há atualização em tempo real).

6. Selo de instalação e controle da pasta oficial

Introduzido na v7 para atender ao requisito de que somente o administrador defina a pasta central e todos entrem apenas nela. O mecanismo contorna uma limitação do navegador: o handle de pasta é intransferível entre máquinas, então não é possível “empurrar” a pasta para os demais — cada pessoa a seleciona uma vez. O que o selo garante é que a seleção só é aceita se apontar para a pasta certa.

Três situações possíveis (variável `situacaoPasta`):

- **oficial:** o `config.json` tem selo e ele confere com o memorizado neste PC — fluxo normal para todos.
- **nao-oficializada** `config.json.instalacao`.
- **divergente:** a pasta tem um selo diferente do já usado neste PC — acesso recusado a todos; só o `adm` pode adotar a nova pasta explicitamente.

Bug corrigido na v8: a oficialização era disparada antes de a identidade global `ident` existir, e `oficializarPasta` tentava ler `ident.nome`, causando “Cannot read properties of null”. A correção passou a

pessoa selecionada como parâmetro (`oficializarPasta(pessoa)`). É um exemplo do cuidado necessário com a ordem de inicialização do estado global.

7. Modelo de segurança e suas fronteiras

Princípio honesto que orienta o projeto: as proteções no aplicativo (senha, papéis, selo) impedem o uso **acidental ou casual** de identidade e permissões alheias, mas **não constituem controle de acesso forte**. Como todo o código roda no cliente e os dados são arquivos numa pasta, uma pessoa determinada com acesso de escrita à pasta pode contornar o app e editar os JSON diretamente.

A barreira real deve ser o sistema de arquivos (NTFS/AD). A recomendação de implantação é configurar permissões da pasta de modo que:

- cada pessoa tenha escrita apenas no próprio arquivo de lançamentos;
- cada chefia de unidade tenha escrita no `aprovacoes.json` da sua unidade;
- a chefia geral tenha escrita apenas na pasta GERAL;
- o administrador tenha escrita em `config.json` (e, se desejada a exclusão administrativa, na árvore toda);
- os demais acessos sejam somente leitura.

Com isso, mesmo que alguém abra o app com um perfil indevido, a gravação falha no nível do sistema operacional e o app exibe o erro. **As senhas do app e as permissões NTFS são camadas complementares:** a senha é o cadeado da gaveta; o NTFS é a fechadura do cofre. Porém, como implementado até a última versão, se as permissões NTFS existirem, ou não, o sistema continua funcionando normalmente. Sem permissões NTFS há um risco maior de erros.

7.1. Senhas

O `config.json` guarda apenas o **hash SHA-256** de `nome + '|' + senha`, calculado no navegador via Web Crypto — a senha em texto nunca é persistida. O administrador gera hashes pelo próprio painel (ou pelo utilitário `gerar-senha.html`). A senha inicial do administrador é `adm` e **deve ser trocada no primeiro acesso**.

8. PWA, execução e implantação

Execução mínima: abrir `index.html` no Chrome ou Edge já habilita todo o sistema, inclusive o acesso à pasta. Os recursos de PWA (instalação com ícone próprio e cache offline via service worker) exigem servir os arquivos por HTTP(S) ou localhost — o navegador não registra service worker a partir de `file://`.

Formas de servir (qualquer uma basta): `python -m http.server 8080`; `npx serve .`; ou publicar a pasta em um servidor de arquivos da intranet. O próprio diretório de rede pode hospedar o app (ex.: `unidadeCentral\app`).

Atualização de versão: o service worker faz cache dos arquivos. Ao publicar mudanças, incrementa a constante `VERSAO` em `sw.js` (atualmente `ra-v8`) para que os navegadores baixem a nova versão. Na primeira carga após atualizar, um recarregamento forçado (Ctrl+F5) ajuda a garantir a troca.

Compatibilidade: Chrome e Edge apenas, por decisão de projeto — a File System Access API não está disponível em Firefox/Safari. Melhorias que ampliem o alcance precisam de um caminho alternativo de persistência (ver seção 11).

9. Limitações conhecidas

- **Sincronização manual:** não há tempo real; a pessoa clica em “Atualizar dados da rede” para ver mudanças de terceiros.
- **Somente Chrome/Edge:** dependência da File System Access API.
- **Seleção da pasta por pessoa:** o handle não é transferível; cada usuário aponta a pasta uma vez por PC.

- **Segurança dependente de NTFS:** sem as permissões do sistema de arquivos, o controle é apenas contra uso casual.
- **Sem auditoria rica:** registra-se “aprovado por/em”, mas não há trilha de auditoria completa nem assinatura.
- **Renomear pessoa desvincula dados:** mudar um nome no catálogo desliga a senha e o arquivo de lançamentos do nome antigo (**o arquivo permanece; cabe ao adm limpar**).
- **Escrita administrativa ampla:** a exclusão de registros por engano é possível; a confirmação reforçada mitiga, mas não impede.

10. Guia prático de modificação

Receitas para as alterações mais prováveis, com o ponto exato de intervenção.

Quero...	Onde mexer
Adicionar/editar processos ou atividades	catalogo.js → array PROCESSOS. Ajustar o campo c (complexidades válidas).
Mudar a pontuação de um nível	catalogo.js → PONTOS. Atenção: lançamentos antigos guardam o valor já calculado.
Adicionar unidade, chefia ou servidor	Em produção: painel do Administrador (grava no config.json). Semente: catalogo.js → UNIDADES.
Alterar o caminho exibido da pasta	catalogo.js → PASTA_REDE.
Mudar regra de quem aprova quem	index.html → funções podeAprovar e aprovacaoDe.
Acrescentar coluna ao CSV	index.html → handler de btnCSV (cabeçalho cab e mapeamento das linhas).
Alterar tema visual	index.html → bloco <style> :root (variáveis de cor) e regras.
Mudar o esquema de um JSON	rede.js (gravação) + index.html (leitura/uso). Manter compatibilidade retroativa.
Forçar nova versão do PWA	sw.js → constante VERSAO.

10.1. Convenções e cuidados

- **Sem build:** o código é servido como está. Não introduza etapas de transpilação sem necessidade — parte do valor do projeto é a **simplicidade** de implantação.
- **Ordem de inicialização do estado global:** variáveis como `ident`, `CFG`, `dirRaiz` e `situacaoPasta` são globais e mutáveis. O bug da v8 nasceu de ler estado antes de ele existir — teste sempre os fluxos de primeira execução (pasta virgem, sem identidade).
- **Compatibilidade de dados:** arquivos JSON já gravados em produção não têm o campo novo que você adicionar. Use valores-padrão defensivos, como faz `papelDe`.
- **Preserve o escritor único:** ao criar uma funcionalidade que grava, pergunte quem é o dono do arquivo. Se dois perfis precisarem escrever no mesmo arquivo, repense o desenho.
- **Teste multiperfil:** para validar o fluxo completo, abra várias abas/janelas apontando para a mesma pasta local de teste e alterne entre servidor, chefia, chefia geral e adm.

11. Roteiro de evolução sugerido

Melhorias ordenadas por relação custo/benefício, preservando a filosofia do projeto quando possível.

11.1. Dentro da arquitetura atual (sem servidor)

- **Exportar no layout oficial (.xlsx):** usar SheetJS no cliente para gerar a planilha no formato que a chefia já conhece, além do CSV atual.
- **Migrar de leitura total para IndexedDB de cache:** reduzir releituras da rede mantendo um cache local com invalidação por atualizadoEm.
- **Verificação de identidade por escrita:** se o NTFS estiver configurado, provar a identidade pela capacidade de gravar no próprio arquivo (aproveitando o login de rede), reduzindo a dependência de senhas.
- **Trilha de auditoria:** registrar em um log append-only (por autor) as ações de aprovação/exclusão, com carimbo de tempo.

- **Bloqueio otimista:** gravar atualizadoEm e detectar sobrescrita entre abas do mesmo usuário.

11.2. Mudança de arquitetura (com backend)

Se os requisitos evoluírem para identidade verificada, auditoria robusta, acesso remoto e concorrência real, a resposta correta passa a ser um **backend mínimo** (ou uma plataforma institucional já existente, como SharePoint ou um sistema de processo eletrônico). O ponto positivo do desenho atual é que **o formulário e o modelo de dados permanecem** — muda apenas a camada de persistência (substituir `rede.js` por chamadas a uma API REST, mantendo os mesmos esquemas JSON). A separação já existente entre a camada de rede e a lógica de interface facilita essa transição.

12. Glossário

Termo	Significado no projeto
PWA	Progressive Web App: site que pode ser instalado e usado offline.
File System Access API	API do navegador que permite ler/gravar arquivos locais/rede com permissão do usuário.
Handle	Referência a uma pasta/arquivo concedida pelo navegador; intransferível entre máquinas.
Selo de instalação	UUID gravado no <code>config.json</code> que identifica a pasta oficial (v7+).
Escritor único	Princípio de que cada arquivo tem um só autor gravando, evitando concorrência destrutiva.
Papel	Função da pessoa (servidor/chefia/geral/adm) que determina permissões e cadeia de aprovação.
Complexidade	Nível (baixa/média/alta) que define a pontuação de uma atividade (5/10/15).
Semente	Dados iniciais em <code>catalogo.js</code> copiados para <code>config.json</code> na primeira oficialização.

ANEXO PWA

O que é uma Progressive Web App?

Uma **Progressive Web App** — **PWA** é uma aplicação desenvolvida com tecnologias da Web — HTML, CSS, JavaScript e um servidor convencional — mas preparada para oferecer algumas características normalmente associadas a aplicativos instalados:

- ícone na tela inicial ou no menu do sistema;
- abertura em uma janela própria;
- funcionamento parcial ou completo sem internet;
- carregamento mais rápido por meio de cache;
- notificações;
- execução de algumas tarefas em segundo plano;
- acesso por URL, como qualquer site.

Portanto, uma PWA não é uma nova linguagem nem um framework. É uma **forma de projetar e configurar uma aplicação web** para que ela se comporte de maneira mais próxima de um aplicativo nativo. ([MDN Web Docs](#))

A ideia central

Uma PWA procura combinar duas categorias de aplicação:

Aplicação web tradicional	Aplicativo nativo
Acessada por uma URL	Instalado no dispositivo
Atualizada diretamente no servidor	Possui ícone próprio
Funciona em diferentes sistemas	Integra-se ao sistema operacional
Não exige loja de aplicativos	Pode operar offline
Normalmente depende da conexão	Pode executar tarefas em segundo plano

A PWA tenta oferecer parte das vantagens das duas:

É um site que pode adquirir comportamento de aplicativo.

Uma única base de código pode atender computadores, celulares e tablets, embora os recursos disponíveis variem conforme o navegador e o sistema operacional. ([MDN Web Docs](#))

Por que o termo “progressive”?

“Progressive” refere-se ao princípio de **aprimoramento progressivo**.

A aplicação deve possuir uma funcionalidade básica acessível como site. Quando o navegador oferece recursos adicionais, a aplicação passa a utilizá-los.

Por exemplo:

1. em um navegador básico, funciona como um site;

2. em um navegador compatível, pode ser instalada;
3. com Service Worker, pode oferecer conteúdo offline;
4. quando permitido, pode enviar notificações;
5. em sistemas compatíveis, pode integrar-se a arquivos, atalhos e compartilhamento.

Assim, a aplicação não deveria deixar de funcionar simplesmente porque determinado recurso de PWA não está disponível. ([MDN Web Docs](#))

Os componentes principais

1. Aplicação web normal

A base continua sendo uma aplicação web comum:

```
HTML + CSS + JavaScript
      ↓
Backend PHP, Python, Java, Node etc.
      ↓
Banco de dados
```

Uma aplicação PHP tradicional, inclusive uma aplicação com várias páginas, pode ser transformada progressivamente em PWA. Ela não precisa ser React, SPA ou usar um framework específico. Tanto arquiteturas SPA quanto MPA podem ser utilizadas. ([web.dev](#))

2. Web App Manifest

O **manifesto** é normalmente um arquivo JSON que descreve como a aplicação deve aparecer quando instalada.

Exemplo:

```
{
  "name": "Sistema de Receitas",
  "short_name": "Receitas",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#294c60",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Ele informa ao navegador dados como:

- nome da aplicação;
- ícones;

- página inicial;
- cor da interface;
- orientação da tela;
- forma de exibição;
- atalhos disponíveis.

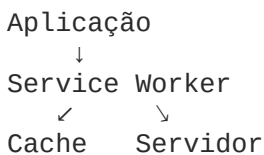
O manifesto é conectado à página HTML:

```
<link rel="manifest" href="/manifest.webmanifest">
```

O arquivo não contém a aplicação. Ele contém **metadados para instalação e integração com o sistema operacional**. (web.dev)

3. Service Worker

O **Service Worker** é um script JavaScript executado separadamente da página. Ele pode ficar entre a aplicação e a rede, interceptando as solicitações realizadas pelo navegador.



Quando a aplicação solicita um arquivo ou informação, o Service Worker pode decidir:

- buscar no servidor;
- devolver uma versão armazenada no cache;
- usar primeiro o cache e depois atualizar;
- tentar a rede e usar o cache em caso de falha;
- apresentar uma página offline.

Isso permite controlar de forma mais inteligente o carregamento e a disponibilidade da aplicação. (web.dev)

Um registro mínimo seria:

```
if ("serviceWorker" in navigator) {
  navigator.serviceWorker
    .register("/service-worker.js")
    .catch(error => {
      console.error("Erro ao registrar Service Worker:", error);
    });
}
```

E um Service Worker bastante simplificado:

```
const CACHE_NAME = "receitas-v1";

const FILES_TO_CACHE = [
  "/",
  "/css/app.css",
  "/js/app.js",
  "/offline.html"
];
```

```
self.addEventListener("install", event => {
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => cache.addAll(FILESTO_CACHE))
  );
});

self.addEventListener("fetch", event => {
  event.respondWith(
    caches.match(event.request)
      .then(cachedResponse => {
        return cachedResponse || fetch(event.request);
      })
  );
});
```

Esse exemplo guarda arquivos essenciais e tenta utilizá-los antes de consultar a rede. Em uma aplicação real, é necessário planejar versionamento, invalidação do cache, autenticação e tratamento dos dados dinâmicos.

4. HTTPS

Service Workers normalmente só são disponibilizados em contextos seguros, ou seja, aplicações servidas por **HTTPS**. Para desenvolvimento local, navegadores geralmente tratam `localhost` como contexto seguro. ([MDN Web Docs](#))

Isso é importante porque o Service Worker pode interceptar comunicações e controlar recursos armazenados localmente.

5. Interface responsiva

Como uma PWA pode ser utilizada em celular, tablet ou computador, sua interface precisa adaptar-se aos diferentes tamanhos de tela.

Entretanto:

Um site responsivo não é automaticamente uma PWA.

Responsividade é uma característica de interface. PWA envolve também instalação, manifesto, cache, Service Workers e integração com recursos da plataforma.

Como funciona para o usuário

Um possível fluxo seria:

1. Usuário acessa `https://sistema.exemplo.gov.br`
2. O navegador carrega a aplicação normalmente.
3. O navegador encontra o arquivo `manifest.webmanifest`.
4. A aplicação registra o Service Worker.
5. Arquivos essenciais são armazenados localmente.
6. O navegador oferece a opção "Instalar aplicativo".
7. O usuário instala.

8. Um ícone é criado no dispositivo.
9. A aplicação pode ser aberta em uma janela própria.
10. Parte do conteúdo continua disponível sem conexão.

A forma de instalação e os recursos oferecidos variam entre navegadores e sistemas operacionais. Nem todos implementam as mesmas funcionalidades de PWA. ([MDN Web Docs](#))

Exemplo aplicado a uma intranet PHP

Considere seu sistema de receitas:

Frontend:
HTML + CSS + JavaScript

Backend:
PHP

Banco:
MySQL ou PostgreSQL

Ele poderia receber uma camada PWA sem substituir o PHP:



O usuário poderia instalar a intranet no computador ou celular. O menu, estilos, ícones e algumas telas poderiam ser armazenados localmente.

Porém, isso não significa que tudo funcionaria offline automaticamente. Operações como:

- autenticar o funcionário;
- verificar permissões;
- consultar receitas atualizadas;
- publicar um livro;

- gravar uma degustação;

continuariam dependendo do servidor, salvo se fosse implementado um mecanismo específico de armazenamento e sincronização offline.

Um cenário offline mais elaborado

Por exemplo, um degustador poderia registrar uma avaliação sem conexão:

1. Avaliação é gravada localmente no IndexedDB.
2. Registro recebe status “pendente de sincronização”.
3. Quando a conexão retorna, a aplicação envia os dados ao PHP.
4. O servidor valida usuário, receita e permissões.
5. Se tudo estiver correto, grava no banco.
6. A aplicação marca o registro como sincronizado.

Nesse caso, seria necessário tratar conflitos, expiração da sessão, alterações simultâneas e tentativas duplicadas.

Estratégias de cache

O cache não deve ser tratado da mesma forma para todos os dados.

Cache first

Consulta primeiro o cache.
Se não encontrar, consulta o servidor.

Adequado para:

- ícones;
- imagens estáticas;
- fontes;
- CSS;
- bibliotecas JavaScript.

Network first

Consulta primeiro o servidor.
Se a rede falhar, usa o cache.

Adequado para:

- lista de receitas;
- dados de funcionários;
- permissões;
- publicações;
- informações que precisam estar atualizadas.

Stale while revalidate

Mostra imediatamente o cache.

Em paralelo, busca uma versão atualizada.

Adequado para:

- imagens de receitas;
- categorias;
- listas de consulta;
- conteúdos cuja pequena defasagem seja aceitável.

PWA não é necessariamente SPA

É comum confundir estes conceitos:

- **PWA:** aplicação web com características progressivas e integração com a plataforma;
- **SPA:** aplicação em que a navegação acontece principalmente dentro de uma única página HTML;
- **MPA:** aplicação formada por várias páginas, normalmente processadas pelo servidor.

Uma aplicação pode ser:

SPA sem ser PWA

MPA sem ser PWA

SPA e PWA

MPA e PWA

Portanto, um sistema PHP tradicional, com páginas processadas no servidor, também pode tornar-se uma PWA. (web.dev)

PWA versus aplicativo nativo

Critério	PWA	Aplicativo nativo
Tecnologias	HTML, CSS e JavaScript	Kotlin, Swift, .NET etc.
Instalação	Navegador ou, em alguns casos, loja	Normalmente loja
Atualização	Publicação no servidor	Frequentemente passa pela loja
Código multiplataforma	Geralmente maior reaproveitamento	Pode exigir projetos separados
Acesso ao dispositivo	Depende das APIs do navegador	Normalmente mais amplo
Funcionamento offline	Precisa ser planejado	Pode ser implementado diretamente
Distribuição por URL	Sim	Normalmente não
Integração com o sistema	Parcial e variável	Mais profunda
Desempenho	Adequado para muitas aplicações	Maior controle para cargas intensivas

Uma PWA costuma ser adequada para sistemas administrativos, catálogos, comércio eletrônico, portais, sistemas internos e aplicações de conteúdo. Aplicações que exigem integração profunda com hardware, execução intensiva ou recursos específicos do sistema podem justificar uma solução nativa.

Síntese

A melhor definição prática seria:

Uma PWA é uma aplicação web que continua acessível como site, mas recebe progressivamente recursos de instalação, cache, operação offline e integração com o dispositivo.

Para transformar uma aplicação existente em PWA, normalmente o caminho inicial é:

Aplicação web existente
+
Design responsivo
+
HTTPS
+
Web App Manifest
+
Service Worker
+
Estratégia de cache e atualização
=
Progressive Web App
